UNIVERSIDAD ANDINA DEL CUSCO

Facultad de Ingeniería y Arquitectura · Escuela Profesional de Ingeniería de Sistemas

Guía 09

Docker y Construcción de Imágenes Seguras en Aplicaciones ASP.NET Core

Dato	Valor
Asignatura	Arquitectura Orientada a Servicios
Semestre académico	2026-2
Ciclo / Categoría	X · Estudios Electivos de Especialidad (EEE)
Docente	Mtro. Ing. Luis Alvaro Monzón Condori
Estudiante	Cuchuirumi Mamani, Manuel Rodrigo
Fecha	11 de noviembre de 2026

Entregables cubiertos

Entregable	Proyecto	Imagen Docker	Puerto
Actividad 01	InventarioClinicoAPI	inventario-clinico-api:v1	8080
Actividad 02	FarmaciaPedidosAPI	farmacia-pedidos-api:v1	8081
Propuesto 01	TriajePacientesAPI	triaje-pacientes-api:v1	8083
Propuesto 02	StockCriticoAPI	stock-critico-api:v1	8082

1. Resumen

Esta guía contenedoriza cuatro APIs ASP.NET Core sobre un escenario clínico: inventario de insumos médicos, pedidos de farmacia, triaje de pacientes y control de stock crítico. Cada una se empaqueta con un Dockerfile de dos etapas y se ejecuta en su propio contenedor.

Las cuatro imágenes se construyeron y los cuatro contenedores quedaron en ejecución simultánea. Todos los endpoints se probaron contra los contenedores, no contra la aplicación ejecutada en local: las capturas de este informe proceden de ese entorno.

2. Entorno de trabajo

La guía parte de una instalación limpia. WSL estaba presente pero con el registro dañado, y el componente «Plataforma de máquina virtual» no estaba habilitado; ambos se repararon antes de instalar Docker Desktop.

Componente	Versión
Sistema operativo	Windows 11 (10.0.26200)
WSL	3.0.5.0 · kernel 6.18.58.1-1
Docker Engine	29.9.3 · contenedores Linux
Docker Desktop	4.99.0
.NET SDK	10.0.401
Imágenes base	mcr.microsoft.com/dotnet/sdk:10.0 y aspnet:10.0

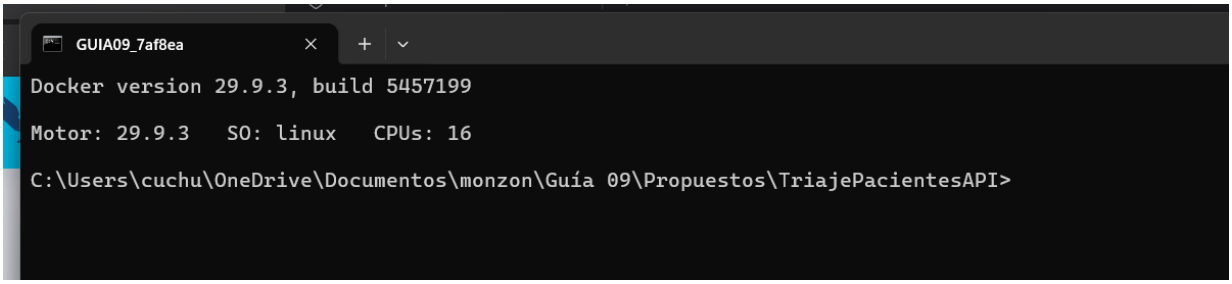


Figura 1. *Versión del motor de Docker y recursos asignados.*

3. Estructura de la solución

```
Guía 09/
  Actividades/
    InventarioClinicoAPI/      Actividad 01
    FarmaciaPedidosAPI/        Actividad 02
  Propuestos/
    TriajePacientesAPI/        Ejercicio propuesto 01
    StockCriticoAPI/           Ejercicio propuesto 02
  capturas/                    evidencias de ejecución
  README.md
```

Cada proyecto es autónomo: su propio archivo de proyecto, su Dockerfile y su .dockerignore. No comparten solución ni dependencias, igual que ocurriría con microservicios desplegados por separado.

4. El Dockerfile

Los cuatro Dockerfiles siguen la misma estructura de dos etapas que plantea la guía, con dos añadidos que el enunciado no pide pero que el título —«imágenes seguras»— justifica.

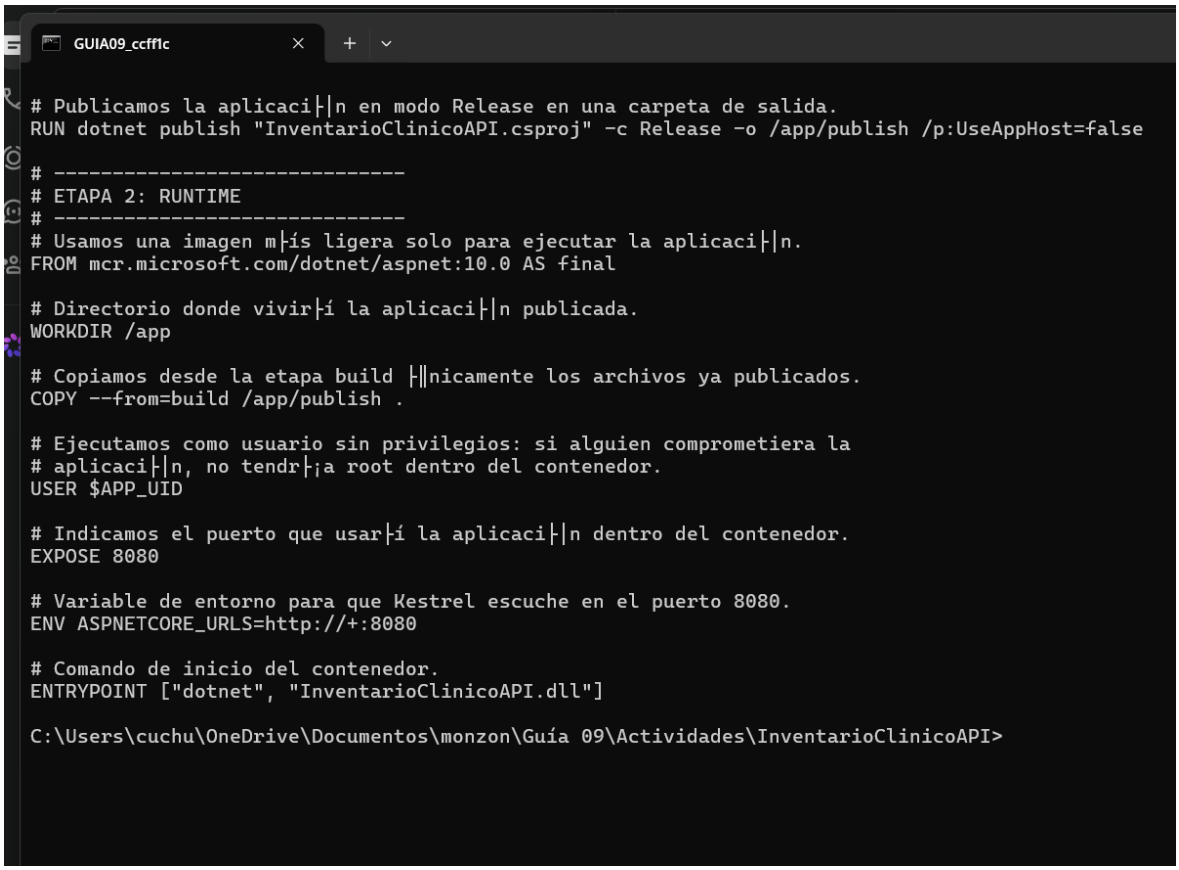


Figura 2. *Dockerfile de la Actividad 01, con sus dos etapas comentadas.*

4.1. Por qué dos etapas

La primera etapa usa la imagen del SDK, que ronda el gigabyte, para restaurar, compilar y publicar. La segunda parte de la imagen de runtime y copia únicamente la carpeta publicada. El SDK no viaja a la imagen final: por eso las imágenes pesan 344 MB y no más de un gigabyte.

La separación también reduce la superficie de ataque. Un contenedor con el SDK dentro lleva compiladores y herramientas que una aplicación en producción no necesita.

4.2. Usuario sin privilegios

La instrucción USER $APP_UID hace que la aplicación se ejecute con un usuario sin privilegios en lugar de root. Es la diferencia práctica entre una imagen que funciona y una imagen segura: si alguien comprometiera el proceso, no tendría control del contenedor.

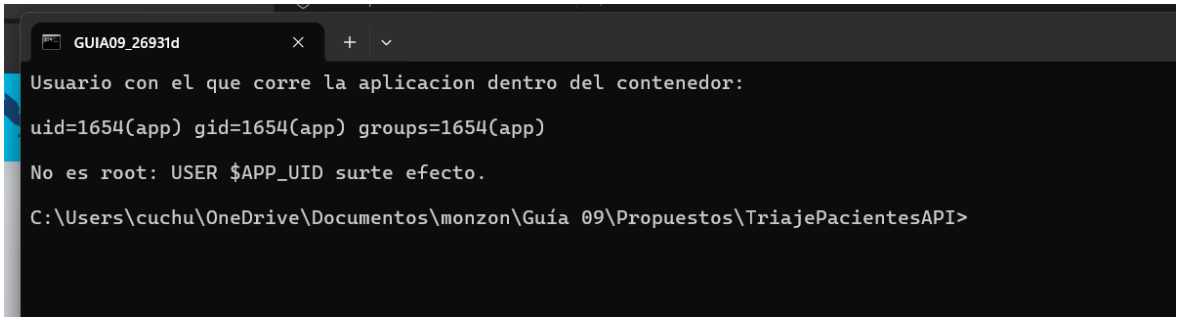


Figura 3. *Comprobación del usuario dentro del contenedor: uid 1654, no root.*

4.3. Archivo .dockerignore

Cada proyecto excluye bin/, obj/ y .vs/ del contexto de construcción. Sin él, Docker copiaría binarios compilados en la máquina anfitriona que la etapa de build va a regenerar de todos modos. Es la buena práctica «no incluir archivos innecesarios» del punto 5.11 de la guía.

5. Construcción de las imágenes

```
docker build -t inventario-clinico-api:v1 .
docker build -t farmacia-pedidos-api:v1 .
docker build -t stock-critico-api:v1 .
docker build -t triaje-pacientes-api:v1 .
```

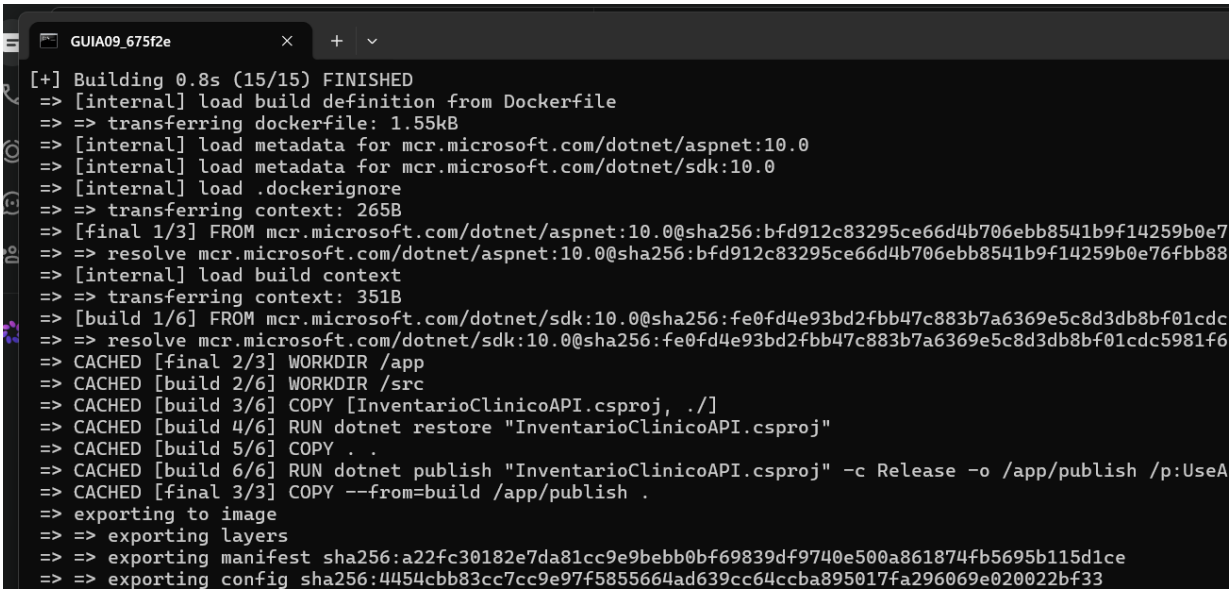


Figura 4. *Construcción de la imagen. Se distinguen las dos etapas: [build 1/6] a [build 6/6] y [final 1/3] a [final 3/3].*

La primera construcción descargó las imágenes base; las tres siguientes reutilizaron esas capas y tardaron segundos. La captura muestra una reconstrucción posterior, donde todos los pasos aparecen como CACHED: Docker reconoce que ni el archivo de proyecto ni el código cambiaron.



Figura 5. *Las cuatro imágenes creadas, de 344 MB cada una.*

6. Ejecución de los contenedores

```
docker run -d -p 8080:8080 --name inventario-api-contenedor inventario-clinico-api:v1
docker run -d -p 8081:8080 --name farmacia-api-contenedor   farmacia-pedidos-api:v1
docker run -d -p 8082:8080 --name stock-api-contenedor      stock-critico-api:v1
docker run -d -p 8083:8080 --name triaje-api-contenedor     triaje-pacientes-api:v1
```

Dentro del contenedor las cuatro escuchan en el puerto 8080; lo que cambia es la correspondencia con el equipo anfitrión. Eso permite ejecutar las cuatro a la vez sin conflicto de puertos, que es justo lo que pide el ejercicio propuesto 02 al exigir un puerto distinto.

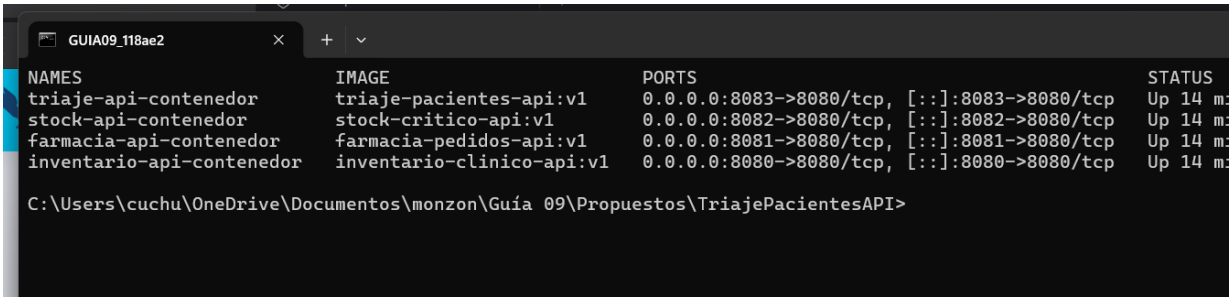


Figura 6. *Los cuatro contenedores en ejecución simultánea.*

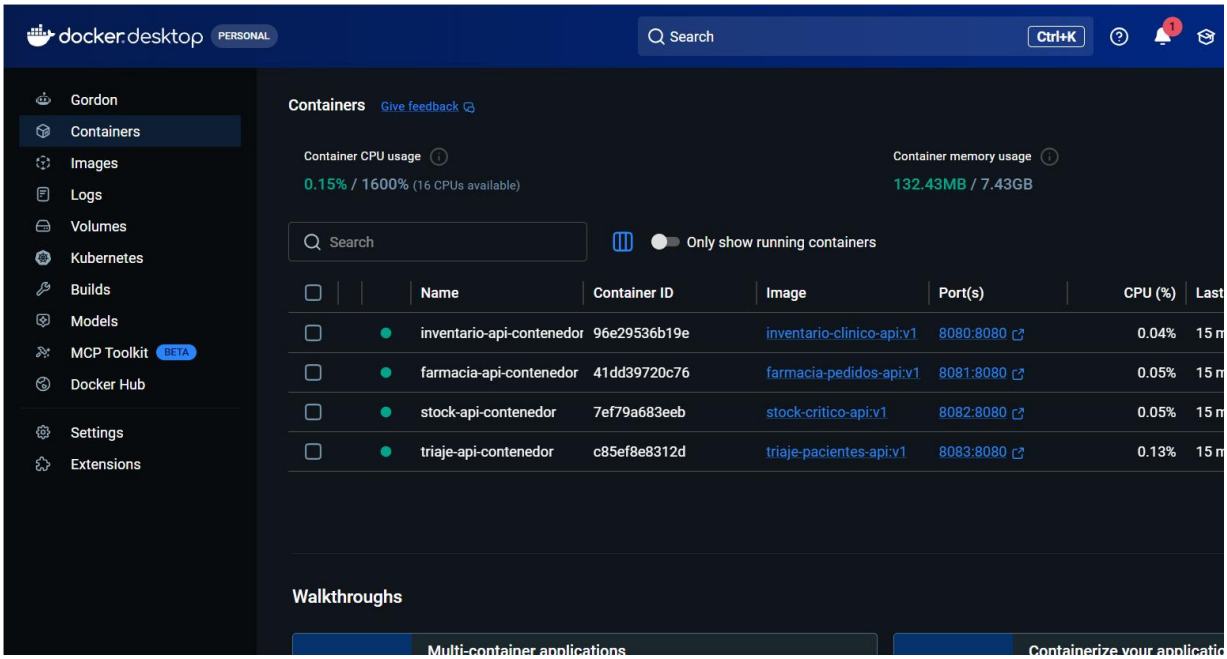


Figura 7. *Los mismos contenedores vistos desde Docker Desktop.*

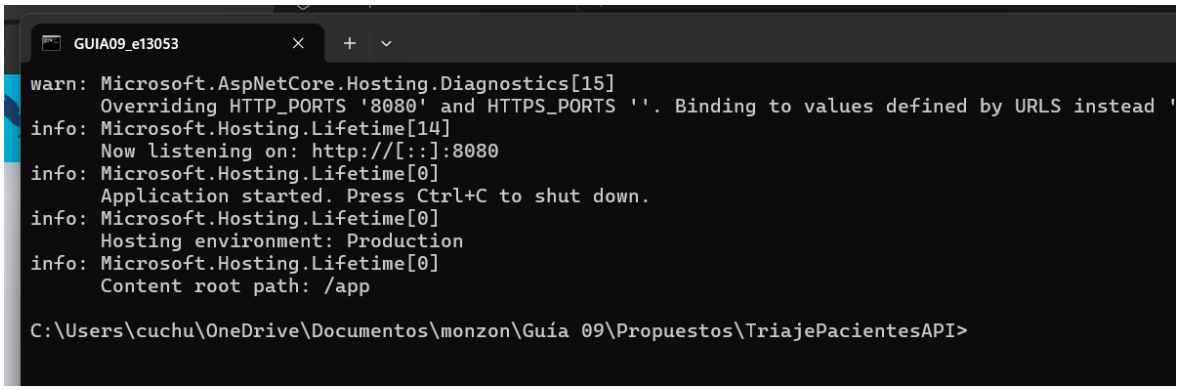


Figura 8. *Registro de arranque del contenedor: Kestrel escuchando en el puerto 8080.*

7. Actividad 01 — Inventario clínico

API de consulta del inventario de insumos médicos. Expone el listado completo y la consulta por identificador, con una respuesta 404 cuando el producto no existe.

Método	Ruta	Respuesta
GET	/api/inventario	4 productos
GET	/api/inventario/3	Alcohol medicinal, stock 180
GET	/api/inventario/99	404 con mensaje explicativo

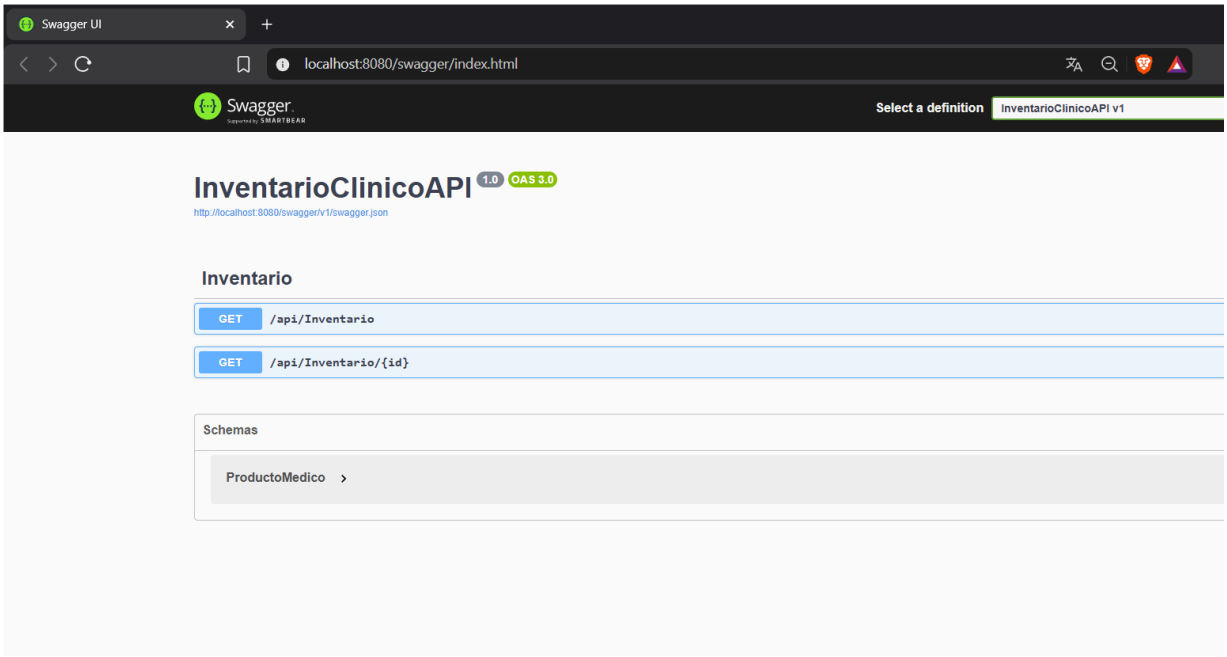


Figura 9. *Swagger de la API de inventario, servido desde el contenedor.*

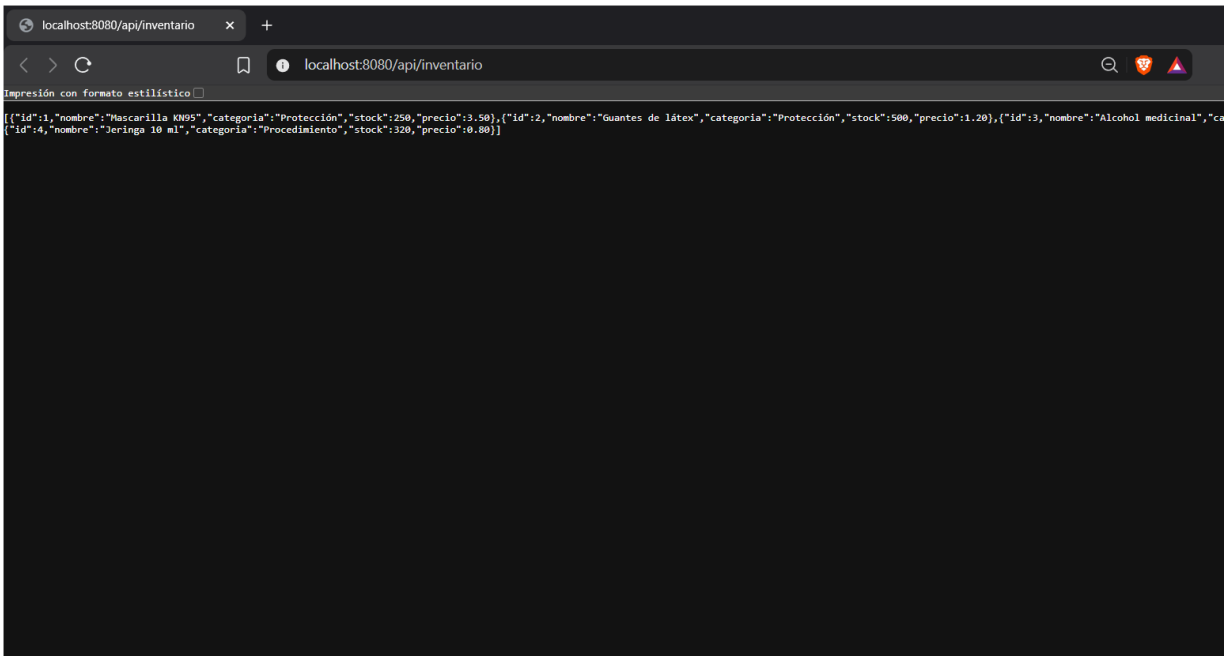


Figura 10. *Respuesta de GET /api/inventario en el puerto 8080.*

8. Actividad 02 — Pedidos de farmacia

Añade el registro de pedidos mediante POST, con validaciones de negocio y asignación automática de identificador, fecha y estado.

Método	Ruta	Comprobación	Resultado
GET	/api/pedidos	Listado inicial	2 pedidos
POST	/api/pedidos	Registro válido	Id 3, estado «Pendiente»
GET	/api/pedidos/3	Consulta del nuevo	Devuelve el pedido
POST	/api/pedidos	Medicamento vacío	400
POST	/api/pedidos	Cantidad igual a 0	400

El pedido con identificador 3 que aparece en la captura no estaba en los datos de ejemplo: se creó mediante POST durante la prueba. La fecha y el estado los asignó el servidor porque la petición no los incluía.

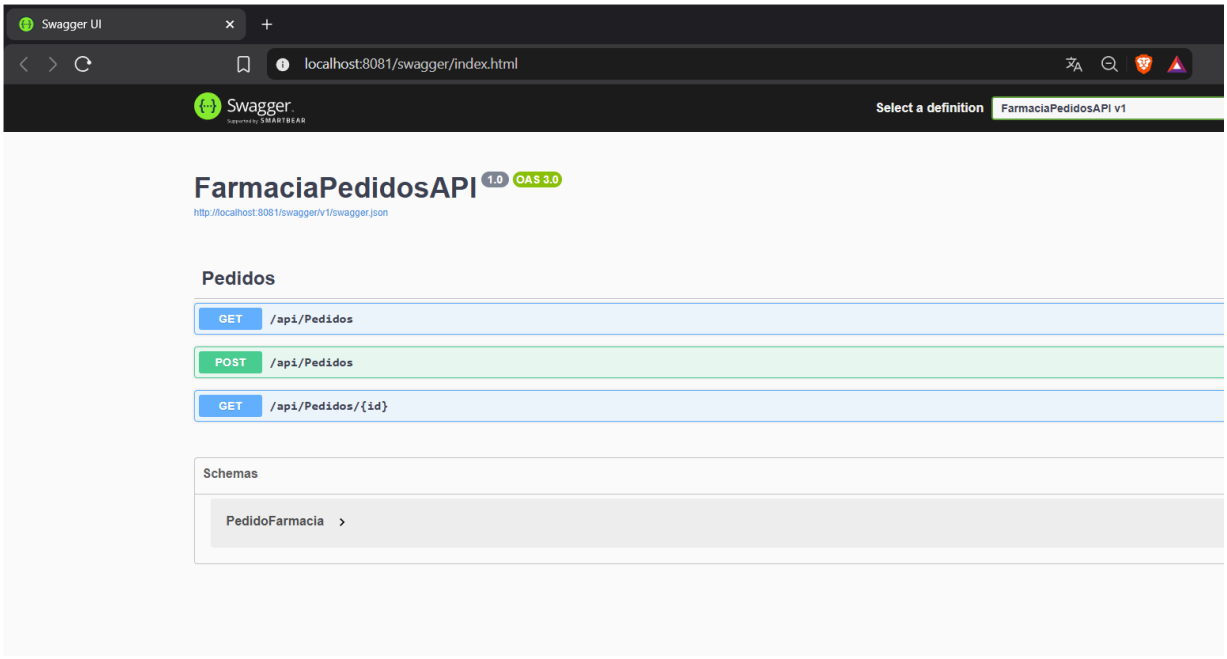


Figura 11. *Swagger de la API de pedidos, con el POST disponible.*

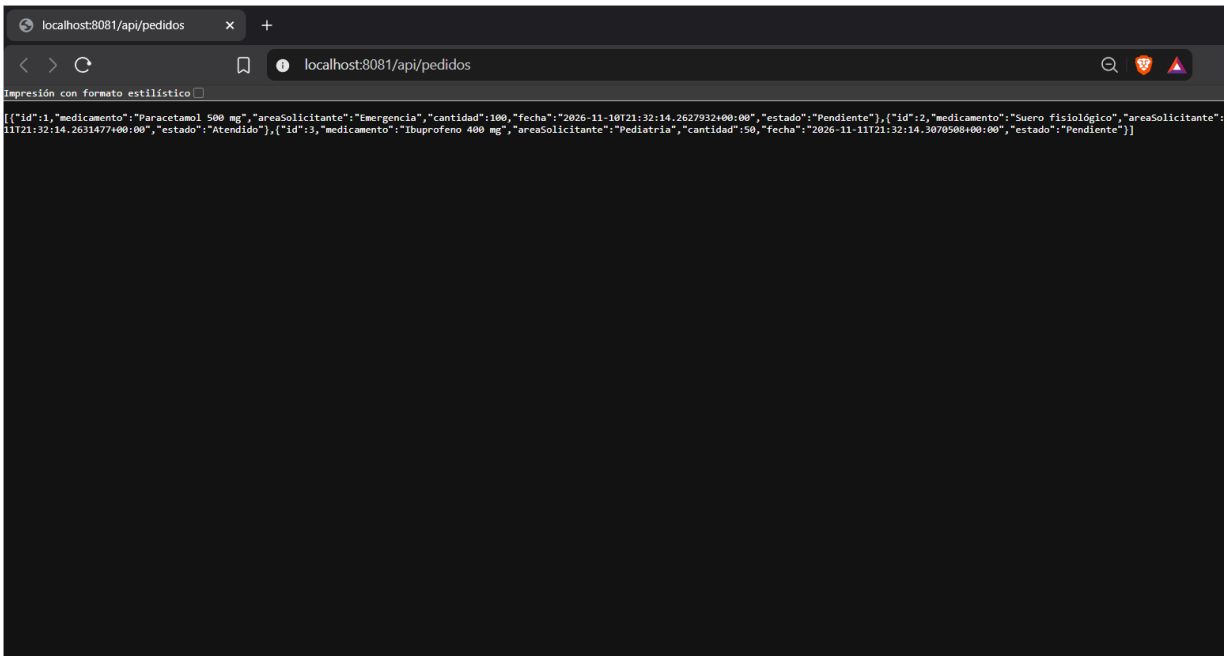


Figura 12. *Listado tras registrar un pedido nuevo.*

9. Ejercicio propuesto 01 — Triaje de pacientes

El nivel de prioridad no lo envía el cliente: lo calcula el servidor a partir de los signos vitales. Cualquier valor que llegue en la petición se descarta, de modo que nadie pueda declararse «Crítico» desde fuera.

9.1. Interpretación del enunciado

La guía enumera dos signos bajo cada nivel sin indicar si deben cumplirse ambos. Se resolvió con o lógico: basta un signo alterado para elevar la prioridad.

Exigir ambos dejaría en «Normal» a un paciente con 40 °C de fiebre y pulso tranquilo, que es precisamente a quien hay que atender primero. Es una decisión documentada, no una omisión.

Nivel	Condición
Crítico	Temperatura > 39 o frecuencia cardíaca > 120
Urgente	Temperatura entre 37 y 39 o frecuencia entre 90 y 120
Normal	Cualquier otro caso

9.2. Casos probados

Temperatura	Frecuencia	Nivel obtenido	Motivo
40,1 °C	80	Crítico	Fiebre sobre el umbral
36,5 °C	150	Crítico	Taquicardia sobre el umbral
37,8 °C	75	Urgente	Febrícula en rango
36,4 °C	100	Urgente	Pulso en rango
36,5 °C	70	Normal	Ambos signos normales

Los dos primeros casos son los que demuestran la interpretación: cada uno tiene un solo signo alterado y ambos se clasifican como críticos.

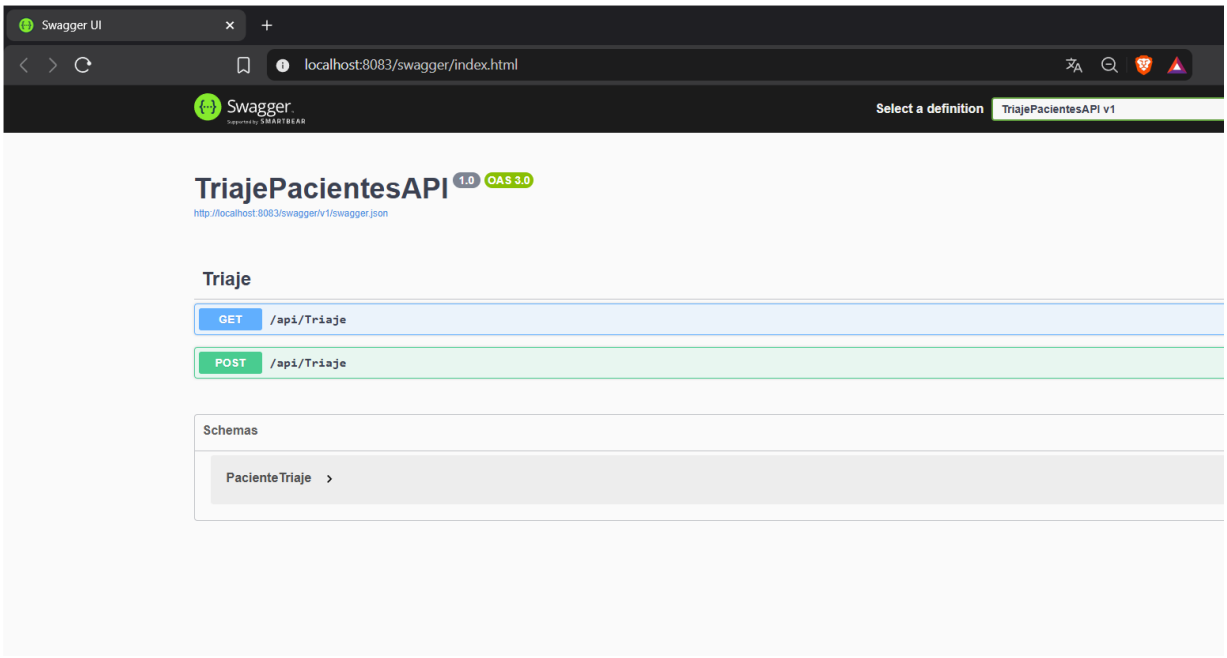


Figura 13. *Swagger de la API de triaje.*

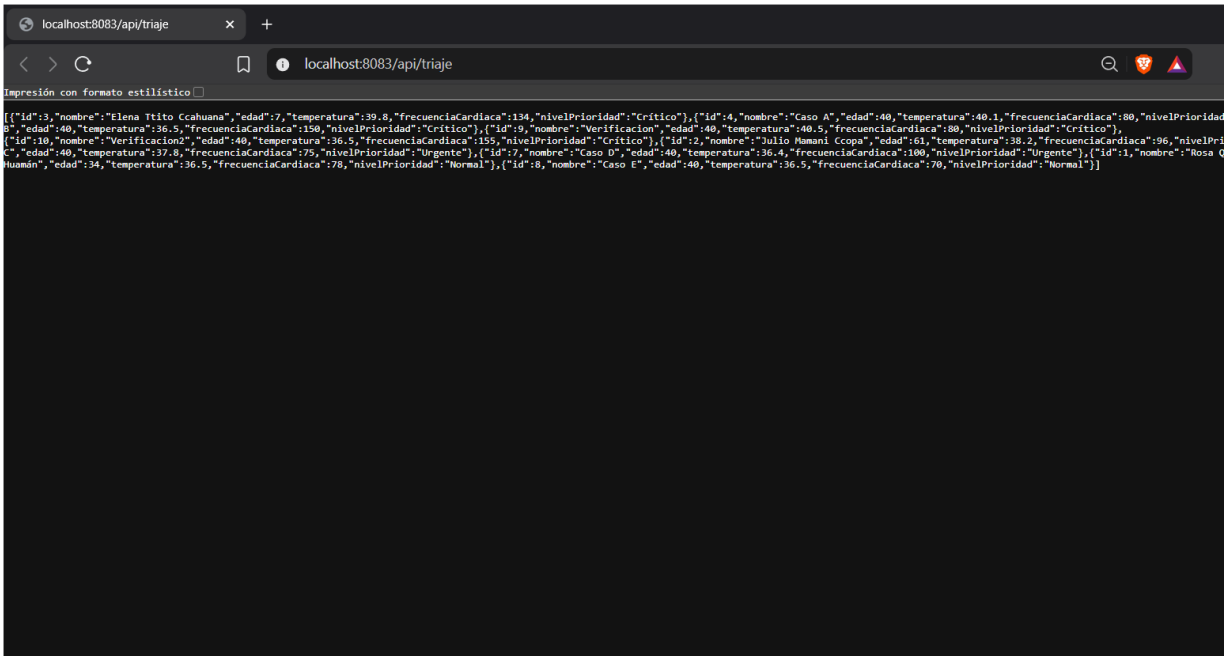


Figura 14. *Listado de pacientes. Aparecen ordenados por gravedad, no por identificador, y con el nivel ya calculado.*

10. Ejercicio propuesto 02 — Stock crítico

El estado es un valor derivado: se recalcula a partir de las existencias y el mínimo cada vez que el producto cambia, de modo que nunca puede quedar desincronizado de las cifras reales.

Estado	Condición
Crítico	Stock < StockMinimo
Alerta	Stock = StockMinimo
Normal	Stock > StockMinimo

Stock	Mínimo	Estado obtenido
5	20	Crítico
30	30	Alerta
90	10	Normal

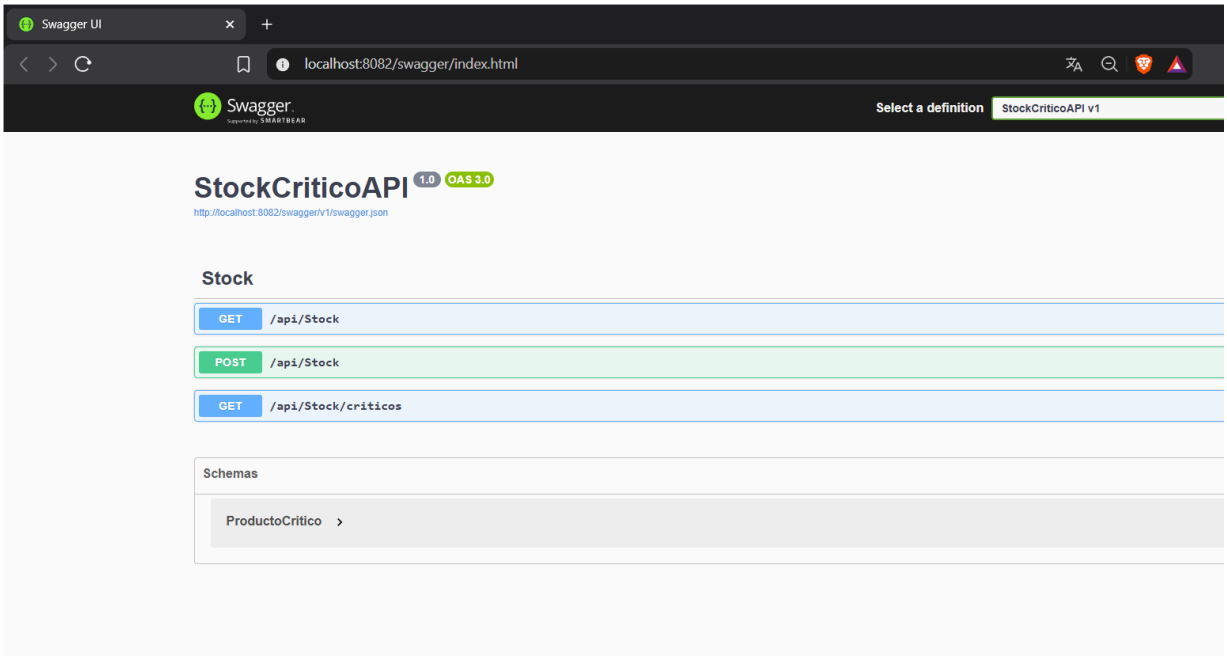


Figura 15. *Swagger de la API de stock, con los tres endpoints.*

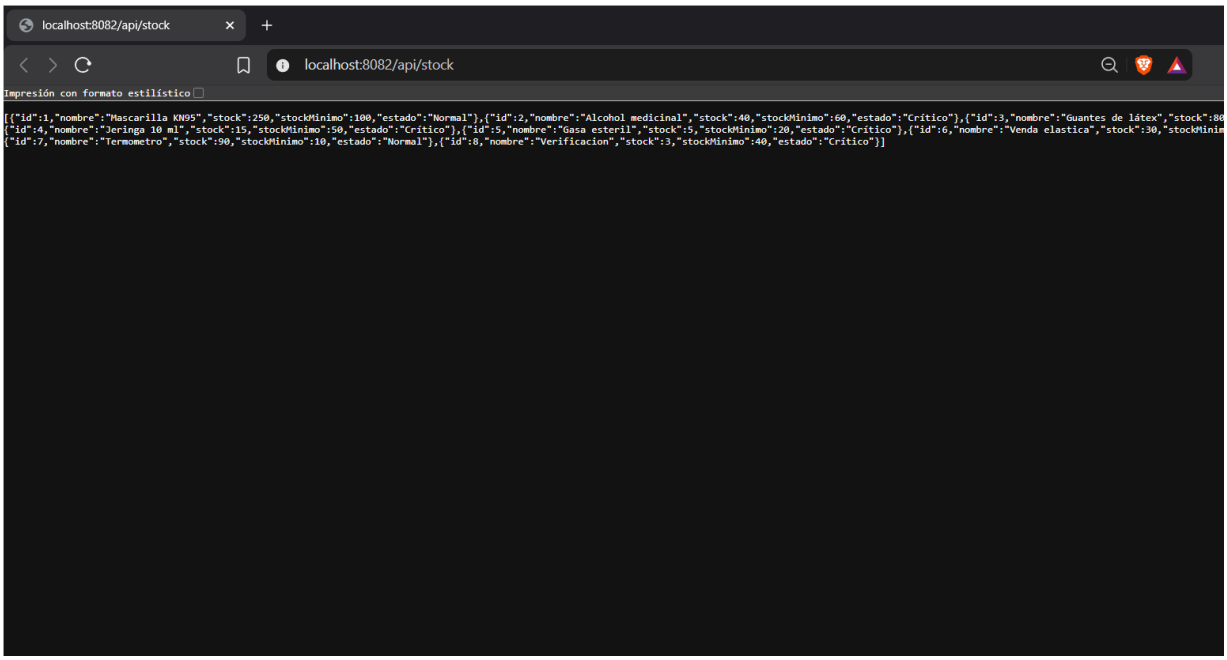


Figura 16. *Listado completo con los tres estados representados.*

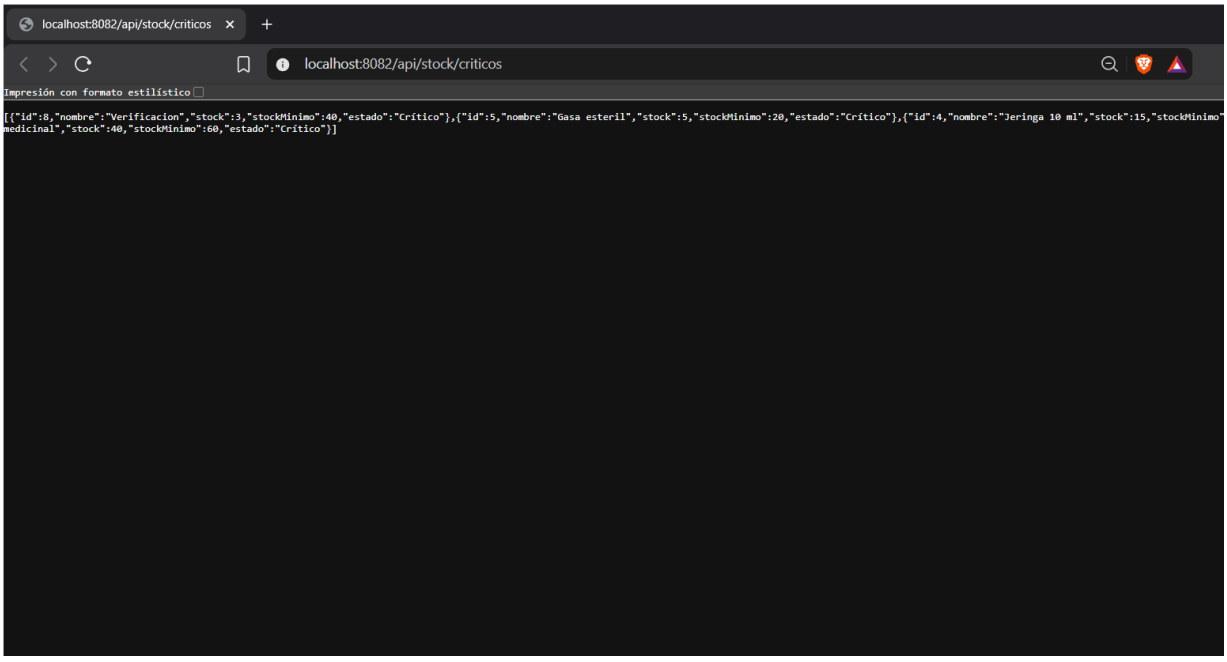


Figura 17. *Endpoint /api/stock/criticos: devuelve solo los productos bajo el mínimo.*

11. Decisiones tomadas

11.1. .NET 10 en lugar de .NET 8

El enunciado pide .NET 8. El equipo solo tiene instalado el SDK de .NET 10, y su plantilla de Web API arrastra paquetes de OpenAPI incompatibles con net8.0. Se usó net10.0 de forma coherente en los cuatro proyectos y en las imágenes base.

La estructura del Dockerfile es exactamente la que enseña la guía. Volver a .NET 8 sería cambiar la etiqueta de las dos imágenes base y el marco de destino del proyecto: tres líneas por proyecto.

11.2. ControllerBase en lugar de Controller

La guía usa Controller en la Actividad 01 y ControllerBase en la Actividad 02. Se unificó en ControllerBase, que es el adecuado para una Web API: Controller añade soporte de vistas que aquí no se utiliza.

11.3. Validaciones añadidas

Los ejercicios propuestos comprueban rangos razonables —edad entre 1 y 120, temperatura entre 30 y 45 grados, frecuencia cardíaca hasta 300, stock no negativo— y responden 400 con un mensaje claro. El enunciado no las exigía, pero una API de triaje que acepta una temperatura de 300 grados no es defendible.

12. Conclusiones

1. La compilación en múltiples etapas reduce la imagen final a 344 MB al dejar el SDK fuera del contenedor de ejecución. El beneficio no es solo de tamaño: la imagen final no lleva compiladores ni herramientas de desarrollo.

2. Ejecutar como usuario sin privilegios cuesta una línea y limita el daño de una intrusión. El Dockerfile de ejemplo de la guía corre como root pese a titularse «imágenes seguras»; corregirlo fue la mejora más rentable del trabajo.

3. Separar el puerto interno del puerto publicado permitió ejecutar cuatro APIs a la vez sin tocar su configuración. Las cuatro escuchan en el 8080 dentro de su contenedor.

4. La lógica calculada en el servidor —prioridad de triaje y estado de stock— no admite que el cliente la imponga. Es la diferencia entre un campo de datos y una regla de negocio.

5. El mayor obstáculo no fue el código sino el entorno: WSL estaba dañado y la plataforma de máquina virtual deshabilitada. Docker depende de esa base, y conviene verificarla antes de dar por hecho que basta con instalar Docker Desktop.

13. Código fuente

Los cuatro proyectos, sus Dockerfiles y las capturas de este informe están publicados en el siguiente repositorio:

https://github.com/Calamar49/Clean-Arq/tree/main/Guia%2009

Para reproducir la guía basta clonar el repositorio, situarse en la carpeta de cada proyecto y ejecutar los comandos de construcción y ejecución indicados en los apartados 5 y 6.

14. Comandos de referencia

```
docker images                               # imágenes creadas
docker ps                                   # contenedores en ejecución
docker logs inventario-api-contenedor       # registro de arranque
docker exec inventario-api-contenedor id    # usuario dentro del contenedor
docker stop inventario-api-contenedor       # detener sin eliminar
docker start inventario-api-contenedor      # volver a ejecutar
docker rm  inventario-api-contenedor        # eliminar el contenedor
docker rmi inventario-clinico-api:v1        # eliminar la imagen
```